

MS Data Engineering

Module 1 — Part II

Machine Learning and Deep Learning for Time Series

Topics covered:

- From classical to machine learning approaches
- Recurrent Neural Networks (RNNs) and BPTT
- Long Short-Term Memory (LSTM) networks
- Gated Recurrent Units (GRU)
- The Transformer and self-attention for sequences
- Practical considerations: training, evaluation, benchmarks

Contents

1	From Classical Models to Deep Learning	2
1.1	Why Neural Networks for Time Series?	2
1.2	Problem Formulation	2
2	Recurrent Neural Networks	3
2.1	The Vanilla RNN	3
2.2	Backpropagation Through Time (BPTT)	3
2.3	Vanishing and Exploding Gradients	3
3	Long Short-Term Memory (LSTM)	4
3.1	The LSTM Cell: Motivation	4
3.2	LSTM Equations	4
3.3	Intuition Behind Each Gate	5
3.4	Why the LSTM Alleviates Vanishing Gradients	5
3.5	Multi-Layer and Bidirectional LSTMs	5
4	Gated Recurrent Unit (GRU)	5
4.1	Architecture	5
4.2	Comparing LSTM and GRU	6
4.3	Practical Training Guidelines for RNN-Based Models	6
5	The Transformer for Sequences	7
5.1	Motivation: From Sequential to Parallel Attention	7
5.2	Scaled Dot-Product Attention	7
5.3	Multi-Head Attention	8
5.4	Positional Encoding	8
5.5	The Transformer Encoder	8
5.6	Complexity and the Long-Sequence Problem	9
5.6.1	PatchTST: a representative modern design	9
5.7	Causal (Decoder) Attention for Forecasting	10
6	Comparative Analysis and Model Selection	10
6.1	Inductive Biases	10
6.2	Empirical Benchmarks	10
6.3	Practical Decision Guide	11
7	Advanced Topics and Recent Directions	11
7.1	Temporal Fusion Transformer (TFT)	11
7.2	N-BEATS and N-HiTS	12
7.3	Foundation Models for Time Series	12
8	Summary	12
A	Mathematical Complements	14
A.1	Backpropagation Through Time: Derivation	14
A.2	Attention as Soft Nearest-Neighbour Retrieval	14
A.3	LayerNorm vs BatchNorm for Sequences	14

1 From Classical Models to Deep Learning

1.1 Why Neural Networks for Time Series?

Classical models (ARIMA, VAR, Kalman) are elegant and well-understood but share structural limitations:

- **Linearity.** The relationship between past and future is assumed linear. Real-world series often exhibit threshold effects, regime switches, and interactions that no linear model captures.
- **Fixed memory.** An ARIMA(p, d, q) model uses a fixed, hand-specified window of history. Neural networks can, in principle, learn to weight any part of a sequence.
- **Univariate focus.** Extending classical models to dozens or thousands of series (a common data-engineering scenario) requires separate models and ad-hoc aggregation.
- **Handcrafted features.** Classical models require choosing p, d, q, s explicitly. Deep networks learn their own features from data.

The deep learning trade-off for time series

Deep models offer greater *expressiveness* — they can approximate any continuous function (by the universal approximation theorem) and learn hierarchical temporal features. The price is: *data hunger*, *computational cost*, *hyperparameter sensitivity*, and *reduced interpretability*. Neither family dominates universally; the right choice depends on the amount of data, the forecast horizon, interpretability requirements, and latency constraints.

1.2 Problem Formulation

Given a historical window $\mathbf{X} = (\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t)$ of length L (the *lookback* or *context length*), learn a function f_θ such that:

$$\hat{\mathbf{x}}_{t+1:t+H} = f_\theta(\mathbf{X}),$$

where H is the *forecast horizon*. The loss is typically mean squared error (MSE) or mean absolute error (MAE):

$$\mathcal{L}(\theta) = \frac{1}{NH} \sum_{n=1}^N \sum_{h=1}^H \ell(\mathbf{x}_{t_n+h}, \hat{\mathbf{x}}_{t_n+h}).$$

Evaluation metrics.

$$\begin{aligned} \text{MAE} &= \frac{1}{H} \sum_{h=1}^H |y_{t+h} - \hat{y}_{t+h}| \\ \text{RMSE} &= \sqrt{\frac{1}{H} \sum_{h=1}^H (y_{t+h} - \hat{y}_{t+h})^2} \\ \text{MAPE} &= \frac{100}{H} \sum_{h=1}^H \left| \frac{y_{t+h} - \hat{y}_{t+h}}{y_{t+h}} \right| \quad (\text{undefined if } y_{t+h} = 0) \\ \text{sMAPE} &= \frac{200}{H} \sum_{h=1}^H \frac{|y_{t+h} - \hat{y}_{t+h}|}{|y_{t+h}| + |\hat{y}_{t+h}|} \end{aligned}$$

2 Recurrent Neural Networks

2.1 The Vanilla RNN

A Recurrent Neural Network processes a sequence step by step, maintaining a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ that summarises the history seen so far.

Definition 2.1 (Vanilla RNN)

Given input $\mathbf{x}_t \in \mathbb{R}^n$ at time t :

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h), \quad (1)$$

$$\mathbf{o}_t = \mathbf{W}_o \mathbf{h}_t + \mathbf{b}_o, \quad (2)$$

where σ is typically $\tanh$ (range $(-1, 1)$), $\mathbf{W}_h \in \mathbb{R}^{d \times d}$, $\mathbf{W}_x \in \mathbb{R}^{d \times n}$, $\mathbf{W}_o \in \mathbb{R}^{m \times d}$.

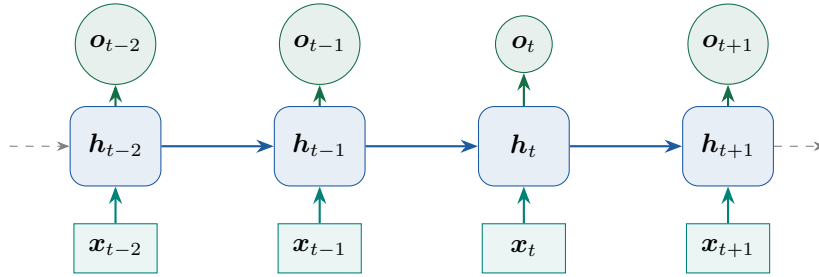


Figure 1: Unrolled vanilla RNN. The same parameters ($\mathbf{W}_h, \mathbf{W}_x, \mathbf{W}_o, \mathbf{b}_h, \mathbf{b}_o$) are shared across all time steps.

2.2 Backpropagation Through Time (BPTT)

To train the RNN, unroll the computation graph across T time steps and apply standard backpropagation. The gradient of the loss with respect to $\mathbf{W}_h$ involves a product of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\sum_{k=1}^t \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_h}.$$

Each term $\partial \mathbf{h}_j / \partial \mathbf{h}_{j-1} = \text{diag}(\sigma'(\cdot)) \mathbf{W}_h$ is a matrix, and the product of $t-k$ such matrices over long sequences leads to the central problem.

2.3 Vanishing and Exploding Gradients

Theorem 2.1 (Gradient behaviour in deep recurrence)

Let $\lambda_{\max}$ be the largest singular value of $\mathbf{W}_h$ and γ a bound on $|\sigma'|$. Then:

$$\left\| \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right\| \leq (\gamma \lambda_{\max})^{t-k}.$$

- If $\gamma \lambda_{\max} < 1$: gradients *vanish* exponentially — the model cannot learn long-range dependencies.
- If $\gamma \lambda_{\max} > 1$: gradients *explode* — training diverges.

Mitigation strategies.

- **Gradient clipping:** rescale gradients if $\|\nabla_{\theta}\mathcal{L}\| > \tau$. This prevents explosion but does not address vanishing.
- **Gated architectures:** LSTM and GRU introduce learnable gates that protect gradient flow — the subject of the next two sections.
- **Truncated BPTT:** propagate gradients only k steps back. Reduces cost but limits the effective memory.

3 Long Short-Term Memory (LSTM)

3.1 The LSTM Cell: Motivation

Hochreiter & Schmidhuber (1997) proposed the LSTM to directly address vanishing gradients. The key innovation is a *cell state* $\mathbf{c}_t$ — a separate memory "conveyor belt" that runs through time with only *additive* updates controlled by three *gates*. Additive updates mean gradients can flow back without being multiplied by a matrix at every step.

3.2 LSTM Equations

Let $\mathbf{x}_t \in \mathbb{R}^n$, $\mathbf{h}_{t-1} \in \mathbb{R}^d$, $\mathbf{c}_{t-1} \in \mathbb{R}^d$. Define concatenated input $\mathbf{z} = [\mathbf{x}_t^\top, \mathbf{h}_{t-1}^\top]^\top \in \mathbb{R}^{n+d}$.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{z} + \mathbf{b}_f) \quad \text{forget gate} \quad (3)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{z} + \mathbf{b}_i) \quad \text{input gate} \quad (4)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c) \quad \text{candidate cell} \quad (5)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{cell state update} \quad (6)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{z} + \mathbf{b}_o) \quad \text{output gate} \quad (7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{hidden state} \quad (8)$$

where σ is the sigmoid function and $\odot$ is element-wise (Hadamard) product.

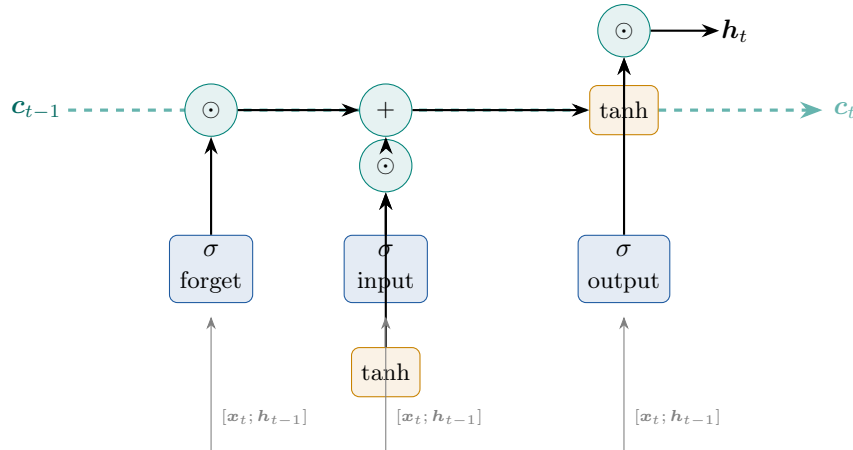


Figure 2: Simplified LSTM cell diagram. The horizontal dashed line is the cell state $\mathbf{c}_t$ — the "information highway" that allows gradients to flow unchanged. The three gates ($\mathbf{f}$, $\mathbf{i}$, $\mathbf{o}$) are all sigmoidal, producing values in $(0, 1)$ that act as soft switches.

3.3 Intuition Behind Each Gate

Forget gate f_t Controls how much of the old cell state $\mathbf{c}_{t-1}$ to retain. $f_{t,j} \approx 1$ means “remember dimension j ”; $f_{t,j} \approx 0$ means “forget it.” Example: when processing a sentence, the forget gate fires on seeing a period, flushing the subject stored in memory for the next sentence.

Input gate i_t Controls how much of the *new* candidate $\tilde{\mathbf{c}}_t$ flows into the cell state. Together with the forget gate, it implements selective writing.

Cell state $\mathbf{c}_t$ Updated *additively*: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. Because gradients of additive nodes are simply summed (not multiplied), the cell state creates an *uninterrupted gradient highway*.

Output gate o_t Decides what part of the cell state to expose as the hidden state $\mathbf{h}_t$ (which feeds the next step and the predictions).

3.4 Why the LSTM Alleviates Vanishing Gradients

Consider the gradient of the loss with respect to the cell state at an earlier time $k < t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{c}_k} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{c}_t} \prod_{j=k+1}^t \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{c}_t} \prod_{j=k+1}^t \mathbf{f}_j.$$

The product is of *gate values* $\mathbf{f}_j \in (0, 1)$, not of weight matrices. If the forget gates learn to stay near 1, the product stays close to 1 and gradients propagate indefinitely. The model effectively learns *when* to let gradients through, rather than suffering passive decay.

Remark

The LSTM is not a perfect cure: gradients through the *hidden state* path still involve matrix products. But the cell-state path provides a gradient shortcut that makes learning long-range dependencies feasible in practice.

3.5 Multi-Layer and Bidirectional LSTMs

Stacked LSTM. Stack L LSTM layers: the hidden state $\mathbf{h}_t^{(\ell)}$ of layer ℓ becomes the input $\mathbf{x}_t^{(\ell+1)}$ of layer $\ell + 1$. Deeper networks can learn more abstract temporal representations.

Bidirectional LSTM (BiLSTM). Run one LSTM forward ($t = 1 \rightarrow T$) and one backward ($t = T \rightarrow 1$), then concatenate their hidden states: $\mathbf{h}_t = [\vec{\mathbf{h}}_t^\top, \overleftarrow{\mathbf{h}}_t^\top]^\top$. Captures both past and future context — useful for sequence labelling (e.g. anomaly detection, named entity recognition), but requires the full sequence and is inapplicable to causal forecasting.

4 Gated Recurrent Unit (GRU)

4.1 Architecture

Cho et al. (2014) proposed the GRU as a simplified gating mechanism with only two gates (vs three for LSTM) and no separate cell state.

Definition 4.1 (GRU equations)

$$\mathbf{r}_t = \sigma(\mathbf{W}_r [\mathbf{x}_t; \mathbf{h}_{t-1}] + \mathbf{b}_r) \quad \text{reset gate} \quad (9)$$

$$\mathbf{u}_t = \sigma(\mathbf{W}_u [\mathbf{x}_t; \mathbf{h}_{t-1}] + \mathbf{b}_u) \quad \text{update gate} \quad (10)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h [\mathbf{x}_t; \mathbf{r}_t \odot \mathbf{h}_{t-1}] + \mathbf{b}_h) \quad \text{candidate hidden state} \quad (11)$$

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \tilde{\mathbf{h}}_t \quad \text{hidden state update} \quad (12)$$

4.2 Comparing LSTM and GRU

	LSTM	GRU
Gates	3 (forget, input, output)	2 (reset, update)
Internal state	Cell $\mathbf{c}_t$ + hidden $\mathbf{h}_t$	Hidden $\mathbf{h}_t$ only
Parameters	$4(d^2 + nd + d)$	$3(d^2 + nd + d)$
Expressiveness	Slightly higher	Slightly lower
Training speed	Slower	Faster (fewer params)
Performance	Often better on long sequences	Often comparable

Reset gate interpretation. $\mathbf{r}_t \approx \mathbf{0}$ forces $\mathbf{r}_t \odot \mathbf{h}_{t-1} \approx \mathbf{0}$, so the candidate $\tilde{\mathbf{h}}_t$ ignores the previous hidden state — effectively *resetting* memory and computing a fresh state from the input alone.

Update gate interpretation. $\mathbf{u}_t$ interpolates linearly between the previous hidden state and the candidate:

$$\mathbf{h}_t = (1 - u_{t,j}) \mathbf{h}_{t-1,j} + u_{t,j} \tilde{\mathbf{h}}_{t,j}.$$

$\mathbf{u}_t \approx \mathbf{0}$: copy the old state (long-range memory). $\mathbf{u}_t \approx \mathbf{1}$: adopt the new candidate (react to new input).

Remark

In practice, LSTM and GRU achieve comparable performance on most time-series benchmarks. The choice is often driven by computational budget: GRU trains about 25% faster and is preferred when the hidden dimension needs to be large for modelling capacity.

4.3 Practical Training Guidelines for RNN-Based Models

- **Normalise inputs** per feature (zero mean, unit variance on the training set). LSTMs are sensitive to input scale because the sigmoid gates saturate for large inputs.
- **Gradient clipping** at norm $\tau \in [1, 5]$ is almost universally beneficial.
- **Dropout** applied to *non-recurrent* connections (Zaremba et al., 2014); for recurrent connections, use *variational (locked) dropout* (Gal & Ghahramani, 2016).
- **Batch size and sequence truncation.** A mini-batch of sequences of length L is standard. Very long sequences require truncated BPTT (typical $L = 50\text{--}200$).

- **Teacher forcing.** During training, feed the ground-truth y_{t-1} as input at step t rather than the model's own prediction. Schedule sampling gradually reduces this assistance during training to close the train/test gap.

Example 4.1 (LSTM for electricity demand forecasting)

Setup: Hourly electricity load at 321 sites (ETTh1 benchmark). Lookback $L = 96$ hours. Forecast horizon $H = 24$ hours.

Architecture: 2-layer LSTM, $d = 256$, dropout 0.1.

Result (MSE): LSTM achieves 0.386 vs ARIMA's 0.621 on the normalised series — a 38% reduction. Particularly large gains on weekends and holidays, where seasonal patterns differ substantially from weekdays and a fixed seasonal period assumption is violated.

5 The Transformer for Sequences

5.1 Motivation: From Sequential to Parallel Attention

Both RNNs and LSTMs are fundamentally *sequential*: computing $\mathbf{h}_t$ requires $\mathbf{h}_{t-1}$, preventing parallelisation across time steps during training. For long sequences (e.g. $L = 512$ or more), this bottleneck is severe.

Furthermore, the hidden state is a *fixed-size vector*: encoding an entire sequence history into d numbers inevitably compresses and loses information. Attention-based models address this by allowing the model to directly access any part of the input sequence.

5.2 Scaled Dot-Product Attention

Definition 5.1 (Query, Key, Value)

Given an input sequence of n vectors stacked as $X \in \mathbb{R}^{n \times d}$, project into three spaces:

$$Q = XW^Q \in \mathbb{R}^{n \times d_k}, \quad K = XW^K \in \mathbb{R}^{n \times d_k}, \quad V = XW^V \in \mathbb{R}^{n \times d_v}.$$

The *attention output* is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The factor $1/\sqrt{d_k}$ prevents the dot products from growing large and pushing the softmax into a near-zero-gradient region.

Interpretation. Each row i of Q is a “query” asking: “which positions in the sequence should I attend to?” The dot product $q_i \cdot k_j$ measures how relevant position j (key) is to position i . The softmax produces a *soft attention weight vector* $\alpha_{ij} \in (0, 1)$ with $\sum_j \alpha_{ij} = 1$. The output for position i is then a weighted average of the values V :

$$\text{out}_i = \sum_j \alpha_{ij} v_j.$$

Example 5.1 (Attention in a time series context)

Consider a 48-hour electricity demand sequence. Position $t = 36$ (noon) needs to predict $t + 12$ (midnight). The attention mechanism may learn to assign high weights

to:

- Last night at midnight (same hour, strong periodic pattern).
- Last Friday midnight (same day-of-week).
- Nearby hours (local smoothness).

Crucially, the model discovers these dependencies from data — no manual seasonality specification is required.

5.3 Multi-Head Attention

Running a single attention function limits the model to one “type” of relevance. Multi-head attention runs H parallel attention heads, each with independent projection matrices, then concatenates and projects:

$$\begin{aligned} \text{MHA}(Q, K, V) &= \text{concat}(\text{head}_1, \dots, \text{head}_H) W^O, \\ \text{head}_h &= \text{Attention}(QW_h^Q, KW_h^K, VW_h^V). \end{aligned}$$

Different heads can capture different aspects simultaneously (local vs global, trend vs seasonality, short-lag vs long-lag dependencies).

5.4 Positional Encoding

Attention is *permutation-invariant*: the output does not change if we shuffle the input positions. For time series, order matters — we must inject positional information.

Definition 5.2 (Sinusoidal positional encoding)

Add a fixed vector $\mathbf{pe}_t \in \mathbb{R}^d$ to each input embedding $\mathbf{x}_t$:

$$\mathbf{pe}_{t,2k} = \sin\left(\frac{t}{10000^{2k/d}}\right), \quad \mathbf{pe}_{t,2k+1} = \cos\left(\frac{t}{10000^{2k/d}}\right).$$

The sinusoidal encoding uses different frequencies for different dimensions, allowing the model to attend to relative positions through linear combinations: $\mathbf{pe}_{t+\delta}$ can be expressed as a linear function of $\mathbf{pe}_t$ for fixed δ .

Learnable positional embeddings (a lookup table, trained end-to-end) are an alternative, though they generalise less well to sequence lengths unseen during training.

5.5 The Transformer Encoder

A single Transformer encoder block consists of:

1. **Multi-head self-attention (MHA):** $Z = \text{LayerNorm}(X + \text{MHA}(X))$
2. **Position-wise Feed-Forward Network (FFN):** $Y = \text{LayerNorm}(Z + \text{FFN}(Z))$, where $\text{FFN}(\mathbf{z}) = W_2 \text{ReLU}(W_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$.

The “Add & Norm” structure creates residual connections that ease gradient flow through very deep stacks (typically 6–12 blocks).

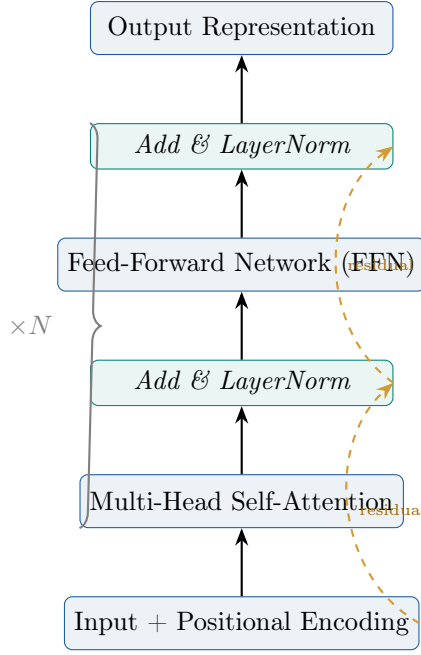


Figure 3: One Transformer encoder block. The full encoder stacks N such blocks (typically $N = 6$). Residual connections bypass each sub-layer, shown as dashed arrows.

5.6 Complexity and the Long-Sequence Problem

Standard self-attention is $\mathcal{O}(n^2 d_k)$ in time and memory — quadratic in sequence length. For short series ($n \leq 512$) this is fine; for long series (hourly data over months: $n = 8760$) it is prohibitive.

Model	Attention complexity	Key idea	Year
Vanilla Transformer	$\mathcal{O}(n^2)$	Full attention	2017
Informer	$\mathcal{O}(n \log n)$	ProbSparse attention	2021
Autoformer	$\mathcal{O}(n \log n)$	Series decomposition + auto-correlation	2021
FEDformer	$\mathcal{O}(n)$	Frequency-domain attention	2022
PatchTST	$\mathcal{O}((n/P)^2)$	Patch-based tokenisation	2023
iTransformer	$\mathcal{O}(n \cdot T^2)$	Attend over variates, not time	2024

5.6.1 PatchTST: a representative modern design

Nie et al. (2023) tokenise the time series into *patches* (non-overlapping or overlapping windows of length P) rather than individual time steps. This reduces the sequence length from L to L/P , drastically cutting the quadratic cost, while preserving local temporal semantics within each patch.

$$\text{Tokens: } [\mathbf{x}_{1:P}, \mathbf{x}_{P+1:2P}, \dots, \mathbf{x}_{L-P+1:L}] \in \mathbb{R}^{(L/P) \times P}.$$

Each patch is linearly projected to d dimensions, positional encodings are added, and a standard Transformer encoder processes the resulting (L/P) -length sequence.

5.7 Causal (Decoder) Attention for Forecasting

For *autoregressive* generation of future values, use a *causal mask*: position t can only attend to positions $1, \dots, t$ (past). This is implemented by setting $\alpha_{ij} = -\infty$ for $j > i$ before the softmax.

$$\text{Causal mask: } M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases} \quad \text{Masked attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

Example 5.2 (Direct multi-step forecasting with Transformer)

Architecture (Informer-style):

- Encoder: $L_e = 96$ input steps, ProbSparse self-attention, 3 layers.
- Decoder: $L_d = 48$ start token + $H = 24$ masked future tokens.
- Output projection: linear layer maps decoder output to predictions.

ETTh1 benchmark, $H = 24$: MSE = 0.302 (Informer) vs 0.386 (LSTM) vs 0.621 (ARIMA). At longer horizons ($H = 720$), the advantage of Transformers over LSTMs widens further.

6 Comparative Analysis and Model Selection

6.1 Inductive Biases

Each model family encodes a different *inductive bias* — the set of assumptions baked into the architecture before seeing any data.

Model family	Inductive bias
ARIMA / VAR	Linearity; stationarity after differencing; Gaussian noise
Kalman Filter	Linear Gaussian dynamics; explicit state decomposition
Vanilla RNN	Nonlinear dynamics; Markovian hidden state; sequential
LSTM / GRU	Selective memory via gating; can learn to ignore irrelevant history
Transformer	Attention to arbitrary positions; permutation-equivariant (without PE)

6.2 Empirical Benchmarks

Empirical results from the M4 and ETT benchmarks (see Table 1 for illustrative figures) suggest several recurring findings:

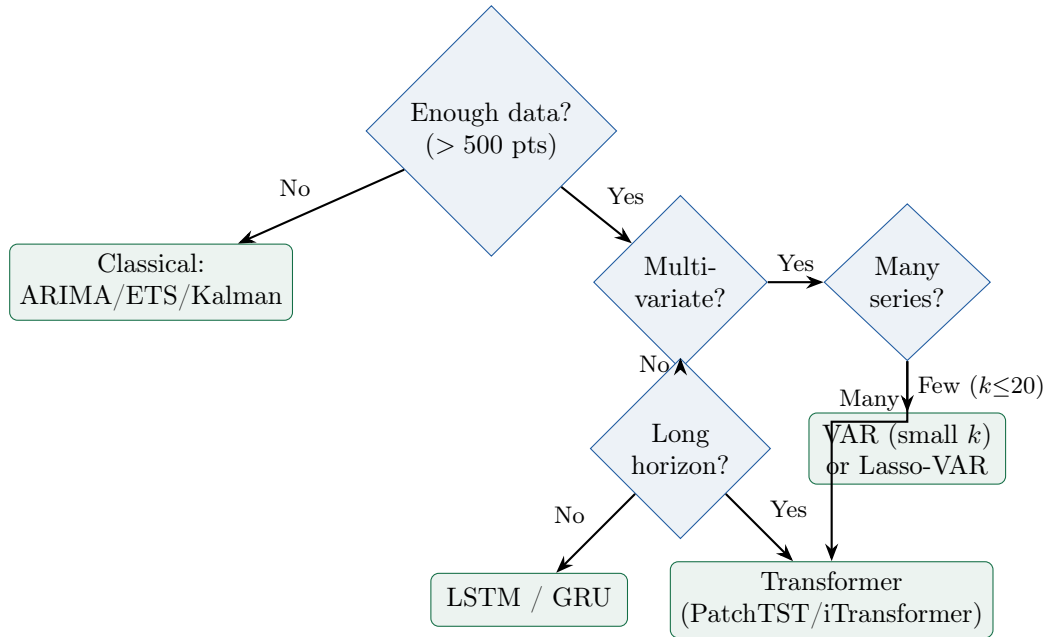
- **Short horizons** ($H \leq 8$): classical methods (Theta, ETS, ARIMA) remain competitive with deep learning.
- **Long horizons** ($H \geq 96$): Transformer-based models consistently outperform LSTM/GRU on benchmark datasets.

- **Simple linear baselines** (DLinear, Nie et al. 2022) have proven surprisingly strong, challenging the assumed necessity of deep architectures for many univariate benchmarks.
- **Multivariate, high-dimensional** settings (hundreds of related series): deep models, especially with cross-series attention (e.g. iTransformer), offer clear advantages.

Table 1: Illustrative MSE comparison on ETTh1 (normalised). Lower is better. Results from published papers; exact values vary with data splits and random seeds.

Model	$H = 24$	$H = 48$	$H = 168$	$H = 720$
ARIMA	0.621	0.784	1.102	1.531
Vanilla LSTM	0.386	0.490	0.738	1.215
GRU	0.374	0.478	0.711	1.178
Informer	0.302	0.411	0.622	0.963
PatchTST	0.270	0.355	0.521	0.841
iTransformer	0.258	0.338	0.498	0.803

6.3 Practical Decision Guide



7 Advanced Topics and Recent Directions

7.1 Temporal Fusion Transformer (TFT)

Lim et al. (2021) propose the TFT for multi-horizon forecasting with mixed static and temporal covariates. Key components:

- **Variable selection networks:** weight the importance of each input variable using gating.
- **Static covariate encoders:** process time-invariant metadata (e.g. store ID, location) and inject it into all temporal layers.

- **Gated Residual Networks (GRN):** adaptively suppress irrelevant components.
- **Interpretable multi-head attention:** heads share values V , so $A = \frac{1}{H} \sum_h \alpha^{(h)}$ can be inspected as “temporal importance” across lags.

7.2 N-BEATS and N-HiTS

N-BEATS (Oreshkin et al., 2020) is a pure deep learning model with no recurrent or convolutional components, built as stacks of *doubly residual* MLP blocks. Each block produces:

1. A *backcast*: partial explanation of the input window.
2. A *forecast*: partial prediction of the horizon.

The final forecast is the sum of all blocks’ forecasts. N-HiTS (Challu et al., 2023) extends this with hierarchical interpolation for multi-resolution modelling.

7.3 Foundation Models for Time Series

Recent work trains very large models on diverse time series corpora, aiming for zero-shot or few-shot forecasting:

- **TimeGPT** (Garza & Mergenthaler-Canseco, 2023): a Transformer pretrained on a large proprietary dataset, available via API.
- **Moirai** (Salesforce, 2024): any-variate, any-frequency foundation model.
- **Chronos** (Amazon, 2024): tokenises time series values and trains a language-model-style Transformer.

These models challenge the traditional paradigm of fitting a new model per dataset, but require careful evaluation to avoid data contamination.

8 Summary

Architectural progression

The field has progressed along two axes:

1. **Memory mechanism:** *Fixed window* (ARIMA) $\rightarrow$ *Hidden state* (RNN) $\rightarrow$ *Gated cell* (LSTM/GRU) $\rightarrow$ *Direct attention* (Transformer).
2. **Modelling scope:** *Univariate, handcrafted* $\rightarrow$ *Multivariate, learned features* $\rightarrow$ *Cross-series, pretrained*.

No single model dominates all settings. Understanding the inductive biases of each architecture is the key to making principled model selections.

Model	Strengths	Weaknesses	Typical use
Vanilla RNN	Simple, fast	Vanishing gradient, poor long-range	Baselines
LSTM	Long-range memory, robust	Sequential, $O(T)$ training	Medium horizons
GRU	Fast LSTM variant	Slightly less expressive	Medium horizons
Transformer	Parallel, long-range, scalable	$O(n^2)$ (vanilla), attention data-hungry	Long horizons, many series

A Mathematical Complements

A.1 Backpropagation Through Time: Derivation

Let $\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$ and $\mathbf{h}_t = \sigma(\mathbf{W}[\mathbf{x}_t; \mathbf{h}_{t-1}] + \mathbf{b})$ (simplified). Define $\delta_t = \partial \mathcal{L} / \partial \mathbf{h}_t$. Then:

$$\delta_t = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \delta_{t+1} \odot \sigma'(\cdot) \cdot \mathbf{W}_h,$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^T \delta_t \odot \sigma'(\cdot) \cdot \mathbf{h}_{t-1}^\top.$$

The recursion for δ_t runs backwards in time, justifying the name “backpropagation *through time*.”

A.2 Attention as Soft Nearest-Neighbour Retrieval

The attention operation $\text{softmax}(QK^\top / \sqrt{d_k})V$ can be seen as a *differentiable key-value lookup*:

- Q : query — what I am looking for.
- K : keys — indices of the memory.
- V : values — content stored at each index.
- Softmax of dot products: soft similarity — all keys contribute, with weight proportional to their similarity to the query.

As $d_k \rightarrow \infty$ with fixed $QK^\top$, the softmax becomes a hard $\arg \max$, recovering exact nearest-neighbour retrieval.

A.3 LayerNorm vs BatchNorm for Sequences

Batch Normalisation normalises across the batch dimension at each time step. This is problematic for sequences because (a) it introduces dependencies across samples in a mini-batch and (b) test-time estimates degrade for variable-length sequences. Layer Normalisation normalises across the *feature* dimension for each sample and each time step independently, making it batch-size-agnostic and the standard choice in Transformers.